10/06/2022

PIPELINING

Pipeline stage times are limited by the slowest resource of computer.

	C_1	C_2	C_3	C_4	C_5	C_6	C_7
I_1	IF	ID	EX	MEM	WB		
I_2		IF	ID	EX	MEM	WB	
I_3			IF	ID	EX	MEM	WB
I_4				IF	ID	EX	MEM WB

Structural hazard: when a planned ins. can't execute in the proper cycle bec. hardware doesn't support the combination of ins.

Eg. single memory instead of ins. and data memory

	C_1	C_2	C_3	C_4	C_5	C_6	C_7	C_8	C_9	
add \$s0, \$t0, \$t1	IF	ID	EX	MEM	WB					3 cycles wasted
sub \$t2, \$s0, \$t3		IF	S	S	S	ID	EX	MEM	WB	total 9 cycles

	IF	ID	EX	MEM	WB	2 cycles wasted
		IF	S	S	ID EX MEM WB	total 8 cycles

forwarding - bypassing

IF	ID	EX	MEM	WB	
	IF	ID	EX	MEM	WB

total 6 cycles

Load-Use Dates Hazard

	C_1	C_2	C_3	C_4	C_5	C_6
lw \$s0, 20(\$t1)	IF	ID	EX	MEM	WB	
sub \$t2, \$s0, \$t3		IF	ID	EX	MEM	WB

IF	ID	EX	MEM	WB	
	IF	ID	S	EX	MEM WB

Control Hazards: When the proper ins can't execute in the proper pipeline clock cycle bec. the ins. that was fetched isn't the one that is needed

- solution: Wait until branch outcome determined before fetching the next instruction

- assumption: We put extra hardware so that we can test registers calculate branch address and update PC during the second stage.

Branch decision and branch set address → ID stage

	C_1	C_2	C_3	C_4	C_5	C_6	C_7
beq $s1, $s2, label	IF	ID	EX	MEM	WB		
or $t1, $t2, $t3		S	IF	ID	EX	MEM	WB

if assumption not applies

IF	ID	EX	MEM	WB	
S	S	IF	ID	EX	

MEMORY

block: Minimum unit of information that can be present

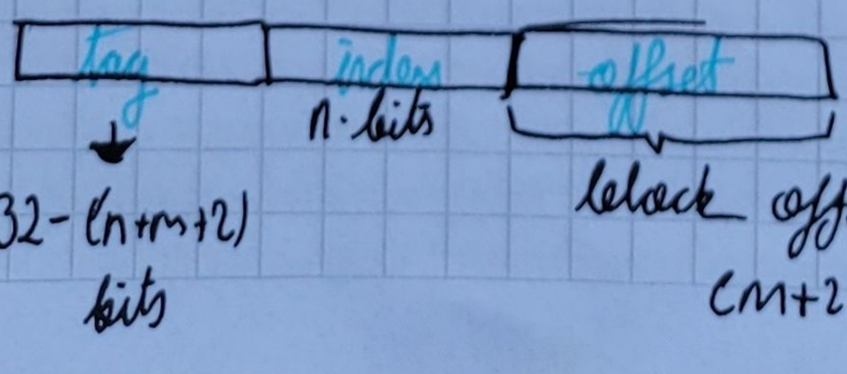
tag: A field is a table used for memory hierarchy that contains the address info required to identify whether the associated block in the hierarchy corresponds to a requested word.

Block = (Block addr) mod (# of blocks in cache)

Block addr = Byte addr / Bytes per block

- Number of bits for a cache

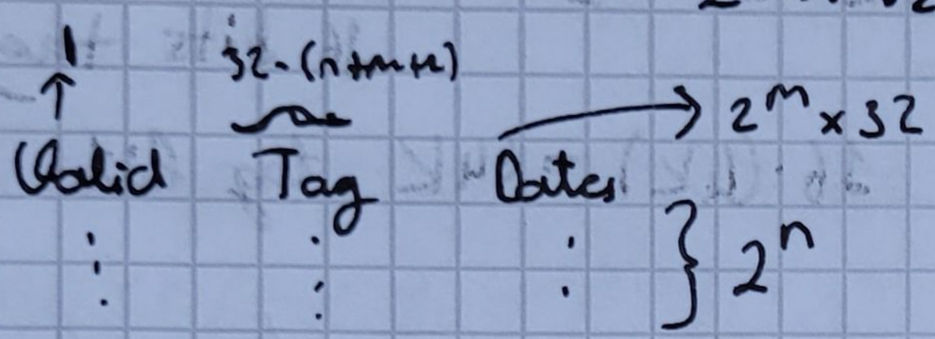
32-bit address, direct-mapped, cache size 2^n blocks, block size 2^m words



20/06/2022

total # of bits is direct mapped cache : $2^n \times$ (block size + tag size + valid field size)

$= 2^n \times (2^m \times 32 + 31 - n - m)$



— Cache Memory

$$\text{Memory stall cycles} = \frac{\text{mem acc.}}{\text{prog}} \times \frac{\text{miss rate}}{1} \times \text{miss penalty}$$

- I-cache: miss rate 2%, miss penalty 100cc
- D-cache: 4%, 100c

Base CPI = 2 LW/SW = 36%

Determine how much faster a proc. would run with a perfect cache? (never miss)

Ins. miss cycle $= I \cdot 2\% \cdot 100 = 2I$

Data miss cycle $= (I \cdot 36\%) \times 4\% \times 100 = 1.44\,I$

$\frac{5.44}{2} = 2.72$

total number of stall cycles: 3.44 total CPI stall = 2 + 3.44 = 5.44

$$\text{AMAT} = \left(\begin{matrix}\text{Hit}\\ \text{time}\end{matrix}\right) \times \left(\begin{matrix}\text{Hit}\\ \text{ratio}\end{matrix}\right) \times \left(\begin{matrix}\text{Miss}\\ \text{time}\end{matrix} \times \begin{matrix}\text{Miss}\\ \text{ratio}\end{matrix}\right)$$

$$\text{Exec time} = \left(\begin{matrix}\text{exec}\\ \text{cycles}\end{matrix} + \begin{matrix}\text{stall}\\ \text{cycles}\end{matrix}\right) \times \text{cycle time}$$

$$\text{stall cycles} = \#\text{ of ins.} \times \text{miss rate} \times \text{miss penalty}$$

direct-mapped: exactly one place

fully-associative: any place (no index bits)

set-associative: middle range

- 4K blocks, 4-word block size, 32-bit address

16 (2^4) bytes per block → 4 bits of offset
32 - 4 = 28 bits for index and tag
Direct mapped : 4K sets $2^2 \cdot 2^{10} = 2^{12}$ → 12 bits index
16 bits tag

$16 \cdot (4K) = 64K$ tag bits

Each degree of associativity decreases number of set by factor of 2.
Also index bits - 1 tag bits + 1

— 2-way set associative
2K sets $2^1 \cdot 2^{10} = 2^{11}$ 11 bits for index
32 - 11 - 4 = 17 bits of tag
$17 \cdot 2 \cdot (2K) = 68K$ tag bits

— 4-way set associative
1K sets $2^0 \cdot 2^{10} = 2^{10}$ 10 bits for index
32 - 10 - 4 = 18 bits of tag
$18 \cdot 4 \cdot (1K) = 72K$ tag bits

— Fully associative cache
1 set with 4K blocks
32 - 4 28 bits of tag
$28 (4K) = 112K$ tag bits

• Cache size = 1024KB Physical memory = 16GB Block size = 32 words

8-way set associative cache

TAG = 15 bits INDEX = 10 bits OFFSET = 7 bits (TOTAL)

32×4 bytes = 128 = 2^7 7 bits offset

1024 KB / 128·8 = 2^{20} byte / $2^7 \cdot 2^3$ = 2^{10} blocks 10 bits index

Tag = 32 - (10+7) 15 bits

— Decrease cache size to 512KB

$\frac{512}{128 \cdot 8} = \frac{2^9 \cdot 2^{10}}{2^7 \cdot 2^3} = 2^9$ 9 bits index

— Decrease block size to 16 words

16×4 = 2^6 6 bits offset

$\frac{1024}{2^6 \cdot 2^3} = \frac{2^{20}}{2^6 \cdot 2^3} = 2^{11}$ 11 bits index

32 - (11+6) = 15 bits tag

— Change to full associativity

32×4 = 128 = 2^7 7 bits offset, 0 bits index, 25 bits tag

— What is the total cache size in part A?

Each entry = data bits + tag bits + valid bit

(32×4×8) + 15 + 1 = 1040 bits

Total size = 1040 × (210 × 8) = 1040 × 213

↓

8-way

